

Задача А. Шестиугольная сетка

Автор задачи: Даниил Орешиников, разработчик: Иван Пальченков

Пропустим в решении **первые две группы тестов**: в них есть очень небольшое количество вариантов, про какие тайлы могут приходить запросы. Достаточно нарисовать картинку на бумаге, вручную посчитать ответ для каждой клетки, после чего перенести эти ответы в ваше решение.

Третья подгруппа уже позволяет посмотреть на задачу более аналитически. Заметим, что можно мысленно разбить плоскость на шесть углов по 60° с вершиной в $(0, 0)$, одним из которых будет угол, ограничивающий тайлы из этой подзадачи: $x_i, y_i \geq 0$. Ключевая идея заключается в том, что поскольку $\vec{A} + \vec{B} + \vec{C} = 0$, до оптимальный путь до любого тайла будет содержать только два из трех разрешенных направлений, и в этой подгруппе это обязательно будут направления A и B .

Перепишем каждое направление как вектор с координатами по x и y . $A = \overrightarrow{(1, 0)}$, $B = \overrightarrow{(-1, 1)}$ и $C = \overrightarrow{(0, -1)}$. Тогда если мы запишем $(x, y) = p \cdot (1, 0) + q \cdot (-1, 1)$, мы получим $x = p - q$ и $y = q$, то есть $q = y$ и $p = x + y$. Иными словами, понадобится сделать $x + y$ шагов в направлении A и y шагов в направлении B . Это и будет ответом для тайлов третьей подгруппы.

В **четвертой группе** тестов $y_i = 0$, и в таком случае достаточно рассмотреть два случая: при $x_i \geq 0$ надо просто сделать x_i шагов типа A , при $x_i < 0$ же — поровну (по $-x_i$) шагов B и C .

Пятая подгруппа решалась полным перебором. Можно было перебрать достаточное количество шагов и на каждом шаге рекурсивно выбирать одно из трех направлений. Альтернативно можно было либо сделать bfs, либо воспользоваться идеей об использовании только двух из трех направлений и перебирать какие два направления будут использоваться и сколько шагов в каждом из них надо будет сделать. Даже если запускать перебор независимо для каждого запроса, такое решение проходило по времени.

Для **полного решения** было достаточно

- либо с помощью формулы определить, в какой трети плоскости находится тайл, после чего применить формулу ниже;
- либо просто перебрать три формулы и выбрать ту из них, в которой количество шагов в каждом направлении получается неотрицательным.

Формулы следующие:

1. если использовать только A и B , то выше мы уже вывели, что число шагов будет $x + y$ и y ;
2. если использовать только A и C , то $(x, y) = p \cdot (1, 0) + q \cdot (0, -1)$, то есть число шагов равно x и $-y$;
3. если же использовать только B и C , то $(x, y) = p \cdot (-1, 1) + q \cdot (0, -1)$, и число шагов будет равно $-x$ и $-x - y$.

Задача В. Странствующий полководец

Автор задачи и разработчик: Павел Скобелкин

Первая подгруппа подразумевала любое решение за малое количество запросов: Например, можно было сдвинуть полководца на вектор $(0, 1)$. Если количество целых точек на отрезке стало равным 2, то эти две точки — это $(0, 0)$ и (x, y) . Иными словами, мы сместились с координатной оси и теперь мы точно знаем, что изначально $y_0 = 0$ (либо мы теперь находимся в точке $(0, -1)$, этот случай можно разобрать отдельно). После этого можно еще раз сдвинуть полководца на вектор $(0, 1)$. Если после этого ответ стал равным 1, то полководец находится в точке $(0, 0)$; иначе загаданная точка имеет координату $(x, 2)$. После этого можно передвинуть точку на вектор $(0, -2)$ — в точку $(x, 0)$. Если полученный ответ уменьшить на один, это и будет координата x .

Вторая подгруппа подразумевала решение перебором: Пусть мы предполагаем, что полководец находится в точке (x, y) . Тогда передвинем его запросом на вектор $(-x, -y)$. Если ответ на этот

запрос 1 — то единственная точка с таким ответом это $(0, 0)$, значит, мы знаем текущее местоположение полководца. Иначе, сделаем другое предположение точки (x', y') . Учитывая то, что сейчас полководец сдвинут на вектор $(-x, -y)$ из-за предыдущего запроса, сдвинем его на $\overrightarrow{(x, y)} - \overrightarrow{(x', y')}$.

Таким образом, мы можем перебрать все предполагаемые точки (x_0, y_0) . В этой подзадаче их не более 81 (все точки в квадрате с углами в $(-4, -4)$ и $(4, 4)$), значит это можно было сделать за $81 \leq 100$ запросов.

Далее будем пользоваться тем, что формально мы можем узнавать ответ в точке, при этом не возвращаясь в исходное положение: если мы хотим узнать ответ в точке $(x_0 + \Delta_1 x, y_0 + \Delta_1 y)$, а после этого в точке $(x_0 + \Delta_2 x, y_0 + \Delta_2 y)$, то первый запрос сделаем для вектора $(\Delta_1 x, \Delta_1 y)$, а второй — для вектора $(\Delta_2 x - \Delta_1 x, \Delta_2 y - \Delta_1 y)$, тогда в обоих запросах мы получим ответ в нужной нам точке.

Третья подгруппа — последняя подгруппа, подразумевающая геометрическое решение.

Решим вспомогательную задачу: Пусть мы знаем, что $|x_0| \leq 42$ и $|y_0| \geq 1000$. Давайте посмотрим ответ в точках $(x_0 - 42, y)$, $(x_0 - 41, y)$, $\dots$ $(x_0 + 41, y)$, $(x_0 + 42, y)$. Так как $|x_0| \leq 42$, то ровно в одной из этих точек будет нулевой x . Если нарисовать это на координатной плоскости, несложно показать, что если $x \neq 0$, то ответ не будет превышать 43, а в точке с $x = 0$ ответ будет превышать 1000. Тогда, перейдя в точку с максимальным ответом, мы окажемся в уже решенном случае с $x = 0$. Итого решение работает за $42 \cdot 2 + 2 = 86$ запросов.

Как же добиться условия $y_0 \geq 1000$? Давайте самым первым запросом передвинем точку с полководцем на вектор $(0, 10^9 + 1000)$. Несложно показать, что при заданных ограничениях $y \geq 1000$ и все еще $|x| \geq 42$. Тогда, за дополнительный запрос мы свели задачу к вспомогательной задаче выше, и решение этой подзадачи работает за 87 запросов.

Для **четвертой подгруппы** нужно было свести геометрическую задачу к алгебраической: несложно показать факт, что количество целочисленных точек на отрезке от $(0, 0)$ до (x, y) это $\gcd(|x|, |y|) + 1$ (где $\gcd(a, b)$ — это наибольший общий делитель чисел a и b , и $\gcd(0, 0) = 0$). Теперь давайте решать другую задачу: загаданы пара чисел (x, y) , и за один запрос можно узнать $\gcd(|x + a|, |y + a|)$. Для начала избавимся от модулей: в самом начале сделаем один запрос, передвинув полководца на вектор $(10^6 + 1, 10^6 + 1)$. Понятно, что после такого трюка координаты точки в любом случае станут положительными.

Рассмотрим битовую запись чисел x и y . Давайте побитово, начиная с конца числа, узнавать биты числа. Каждый шаг будет выглядеть следующим образом:

1. Сделаем запрос парой чисел $(0, 0)$. Если ответ на запрос (то есть $\gcd(x, y)$) кратен двум, значит оба числа кратны двум. Значит, последний бит обоих чисел равен 0, в таком случае перейдем к следующему шагу.
2. Если же ответ на запрос не кратен двум, значит хотя бы у одного числа последний бит 1: давайте сделаем запрос $(1, 0)$. Если ответ на этот запрос кратен двум, значит теперь у обоих чисел последний бит 0 (и до запроса у числа x был последний бит 1, а у числа y — последний бит 0), перейдем к следующему шагу.
3. Если нет, то осталось всего два варианта последних битов: или $(0, 1)$ или $(1, 1)$ у x и y соответственно. Учитывая, что мы уже сдвинули x на 1, а y на 0, для проверки $(0, 1)$ нужно сделать запрос с вектором $(-1, 1)$, а после этого — $(0, 1)$. Таким образом, мы перебрали всевозможные варианты последнего бита у чисел, и с помощью первого шага занулили последний бит чисел.

Сделаем так 20 шагов, каждый раз двигаясь на бит влево: для этого нам нужно делать те же действия, только проверять делимость $\gcd$ на нужную степень двойки — 2^k для k -го бита справа.

Таким образом, не более чем за четыре запроса можно занулить последний бит обоих чисел. Тогда, если $|x_0|, |y_0| \leq 2 \cdot 10^6$, то в битовой записи чисел не больше 21 знака, значит за $21 \cdot 4$ запроса можно превратить числа к виду $100 \dots 00$ в двоичной системе, то есть после всех этапов будет $x = y = 2^{21}$.

Решение **пятой подгруппы** — оптимизация решения для предыдущей подгруппы. Давайте заметим, что четвертый запрос делать нет смысла (так как мы уже точно знаем биты x и y , ведь это единственный оставшийся вариант), значит можно совместить последний запрос передвижения с

первым запросом следующего шага. Тогда теперь можно увеличить количество этапов до 31, потратив в каждом этапе по три запроса. Таким образом, после 31 этапа достаточно вывести $x = y = 2^{32}$.

Задача С. Торговля

Автор задачи и разработчик: Павел Скобелкин

Первая подгруппа подразумевала достаточно прямолинейное решение полным перебором. Ограничения в ней были небольшие, так что эту подгруппу должна была пройти любая аккуратная написанная идея полного перебора.

Во **второй подгруппе** нужно было заметить, что если и приводить все товары к одной цене, то выгоднее всего приводить все s_i к максимальному.

В **третьей подгруппе** при $k = 2$ одна из полученных цен все еще должна быть равна максимальной s_i , а вторую можно было перебрать — назовем перебираемую цену s_j . Теперь все товары, цены которых меньше либо равны s_j , из жадных соображений должны быть приведены ровно к s_j , а все остальные — к $s_{\max}$. Асимптотика полученного решения $\mathcal{O}(n^2)$.

В **четвертой подгруппе** нужно было заметить, что из тех же жадных соображений необходимо приводить к одинаковой стоимости товары, идущие подряд в отсортированном массиве цен. То есть, после сортировки массива изначальных цен, мы должны разбить его на k отрезков, и в каждом из отрезков решить задачу для $k = 1$, то есть сделать все элементы равными максимальному (самому правому из них). Теперь, предположив префиксные суммы на массиве s , можно быстро считать стоимость, необходимую для приведения подотрезка массива s к одной стоимости: пусть необходимо посчитать ответ на отрезке с i по j . Тогда стоимость этого:

$$c(i, j) = \sum_{l=i}^j (s_j - s_l) = s_j \cdot (j - i + 1) - \sum_{l=i}^j s_l$$

Тогда для этой подзадачи можно было перебрать два индекса i и j , и разбить массив на три части с помощью этих двух индексов.

Пятая подгруппа подразумевает обобщение предыдущего решения на динамическое программирование: давайте положим $\text{dp}_{i,j}$ как минимальную стоимость приведения префикса длины i массива s (отсортированного, как указано ранее) к j различным элементам. Тогда пересчет ДП за $\mathcal{O}(n)$ тривиален (p_i — префиксная сумма первых i элементов):

$$\text{dp}_{i,j} = \min_{1 \leq l < i} (\text{dp}_{l,j-1} + c(l+1, i)) = \min_{1 \leq l < i} (\text{dp}_{l,j-1} + s_i \cdot (i - l) - (p_i - p_l))$$

ДП имеет $\mathcal{O}(nk)$ состояний, пересчет каждого работает за $\mathcal{O}(n)$, итоговая асимптотика решения подгруппы $\mathcal{O}(n^2k)$.

Для решения **шестой подгруппы** в пересчете ДП нужно было заметить, что размеры блоков не превышают $n - k$. Тогда в ДП, описанном выше, нет смысла брать $l > n - k$, значит пересчет одного состояния работает за $\mathcal{O}(n - k)$, итоговая асимптотика $\mathcal{O}(nk(n - k))$, что не сильно меньше, но является достаточной оптимизацией.

Полное решение подразумевает оптимизации ДП, описанного выше. На самом деле, эта задача имеет довольно много решений разными оптимизациями ДП (разделяй и властвуй, Convex Hull Trick, λ -оптимизация). Опишем решение через СНТ:

Рассмотрим и перегруппируем члены в пересчете динамики:

$$\text{dp}_{i,j} = \min_{1 \leq l < i} (\text{dp}_{l,j-1} + s_i \cdot (i - l) - (p_i - p_l)) = \min_{1 \leq l < i} (\text{dp}_{l,j-1} + s_i \cdot (i - l) - p_i + p_l) = C + \min_{1 \leq l < i} (\text{dp}_{l,j-1} - l \cdot s_i + p_l),$$

где C — некоторая константа, не зависящая от l . Тогда под минимумом находится линейная функция, коэффициенты которой убывают ($-l$ убывает при возрастающем l), значит в данном случае можно воспользоваться СНТ, если пересчитывать динамику по слоям второго измерения.

То есть сначала посчитаем слой с $j = 1$, далее $j = 2, \dots, j = k$. Таким образом, пересчет одного слоя динамики работает за $\mathcal{O}(n \log(n))$, суммарная асимптотика решения — $\mathcal{O}(nk \log(n))$.

Задача D. Дерево навыков

Автор задачи и разработчик: Ильнур Валеев

Для небольших ограничений **первой и второй подгрупп** можно использовать простое моделирование.

- для каждой вершины, выбранной в качестве корня, храним в каждой вершине отсортированный массив типов навыков;
- перебираем все пары вершин и сравниваем их массивы;
- для каждой вершины сохраняем ответ.

Асимптотика такого решения составляет $\mathcal{O}(n^3)$.

В **третьей группе тестов** используется оптимизация с подсчётом частот типов навыков. В каждой вершине будем хранить массив частот длиной 3 (так как $c_i \leq 3$), который также однозначно задаёт мультимножество навыков в поддереве, как и их отсортированный массив. Иными словами, для каждого типа навыка храним количество его вхождений в поддерево. Два поддерева считаются похожими, если массивы частот совпадают.

Для подсчёта числа похожих пар будем поддерживать ассоциативный массив (`map`, словарь) `cnt`, где ключ — массив частот, а значение — количество вершин, для которых массив частот равен ключу. Тогда ответ вычисляется как $\sum_{(k,v) \in \text{cnt}} C_v^2$.

Ключевая идея заключается в *переподвешивании за соседнюю с корнем вершину*. Пусть мы посчитали ответ для случая, когда изначальный корень дерева зафиксирован. Заметим, что при переподвешивании дерева за ребенка корня можно быстро пересчитать ответ: ровно два поддерева с вершинами в старом и новом корне изменятся, а тогда сумму выше можно пересчитать за $\mathcal{O}(1)$ или хотя бы за время, пропорциональное числу соседей старого корня.

Тогда:

- изначально считаем ответ для корня;
- рекурсивно (с помощью `dfs`) переходим к другим вершинам, пересчитывая значения в `cnt`, а также обновляя ответы.

В конце обхода каждая вершина побывает корнем дерева и мы будем знать все ответы. Асимптотика решения — $\mathcal{O}(n \cdot \max c_i)$.

Для увеличения числа типов навыков ($c_i \leq 20$) в **четвертой группе тестов** можно использовать хеширование массивов:

- массив частот каждого поддерева хешируется в полиномиальный хеш;
- При переподвешивании дерева пересчитывается хеш каждого поддерева.

Объединять массивы можно с помощью оптимизации «small-to-large»: всегда при слиянии добавляем элементы меньшего в больший, что даст асимптотику решения $\mathcal{O}(n \log n)$.

Для **полного решения** достаточно было применить хеширование типов навыков с помощью коммутативной функции. Подойдет любая хеш-функция, устойчивая к перестановкам — таких существует много, примеры можно найти, прочитав про хеширование множеств и мультимножеств. **Пятая подгруппа** отличается тем, что можно было придумать хорошую хеш-функцию и не придумывать идею переподвешивания и предподсчета.

Коммутативная функция позволяет ускорить процесс пересчета хешей для всех переподвешиваний до $\mathcal{O}(n \log n)$ в сумме.